

**LAPORAN PRAKTIKUM**  
**POSTTEST 6**  
**ALGORITMA PEMROGRAMAN LANJUT**



**Disusun oleh:**  
**Hamam Syamil (2509106073)**  
**Kelas B2'25**

**PROGRAM STUDI INFORMATIKA**  
**UNIVERSITAS MULAWARMAN**  
**SAMARINDA**  
**2025**

## 1. Flowchart

### 1. Struktur Data & Inisialisasi

Program menggunakan Struct (User, Penyewa, Kost) untuk mengelompokkan data secara teratur dalam Array of Struct. Data penyewa dikelola sebagai nested struct di dalam data kost untuk keterhubungan informasi.

### 2. Menu Awal (Autentikasi)

Program menyediakan pilihan Register dan Login. Fungsi registerUser bertugas menambah akun dengan validasi kapasitas, sementara loginUser memverifikasi akun dengan batas tiga kali percobaan. Jika login gagal total, program berhenti; jika sukses, pengguna diarahkan ke menu utama.

### 3. Menu Utama (Operasi CRUD)

Menu ini dikelola dalam prosedur menuUtama menggunakan perulangan. Program menyediakan fitur:

- Menggunakan fungsi tambahData untuk input
- Prosedur lihatData untuk menampilkan tabel.
- Fungsi updateData yang memanfaatkan fungsi rekursif cariKamar untuk menemukan ID yang tepat sebelum data diubah.
- Fungsi hapusData memanfaatkan fungsi rekursif cariKamar untuk menemukan ID yang tepat sebelum data dihapus

### 4. Fitur Tambahan & Ringkasan

Terdapat fitur Ringkasan yang menggunakan fungsi rekursif totalHarga untuk menghitung pendapatan secara otomatis. Selain itu, program menerapkan Function Overloading pada prosedur tampilInfo, sehingga satu nama fungsi bisa menampilkan detail kamar dalam berbagai format sesuai kebutuhan.

### 5. Implementasi Pointer & Reference

#### - Pass by Reference (&)

Digunakan pada parameter fungsi pengelola data (seperti tambahData, hapusData, dan registerUser) agar perubahan jumlah data langsung tersinkronisasi dengan variabel asli di fungsi main tanpa perlu melakukan penyalinan data.

- **Pointer pada Struct (\*)**

Digunakan dalam prosedur tampilInfo untuk mengakses anggota *struct* melalui alamat memorinya. Hal ini menunjukkan kemampuan program dalam melakukan manipulasi data pada tingkat memori (low-level) untuk efisiensi eksekusi.

## 6. Algoritma Pengurutan (Sorting)

Program telah dilengkapi dengan menu pengurutan data untuk memudahkan manajemen informasi kost. Terdapat tiga algoritma berbeda yang diimplementasikan:

- **Bubble Sort**

Mengurutkan nama penyewa dari A ke Z dengan membandingkan elemen yang berdekatan secara berulang.

- **Selection Sort**

Mengurutkan harga kamar dari yang tertinggi ke terendah dengan mencari nilai maksimal di setiap iterasi dan menempatkannya di posisi yang tepat.

- **Insertion Sort**

Mengurutkan ID kamar secara menaik dengan cara menyisipkan setiap elemen ke posisi yang benar dalam bagian array yang sudah terurut.

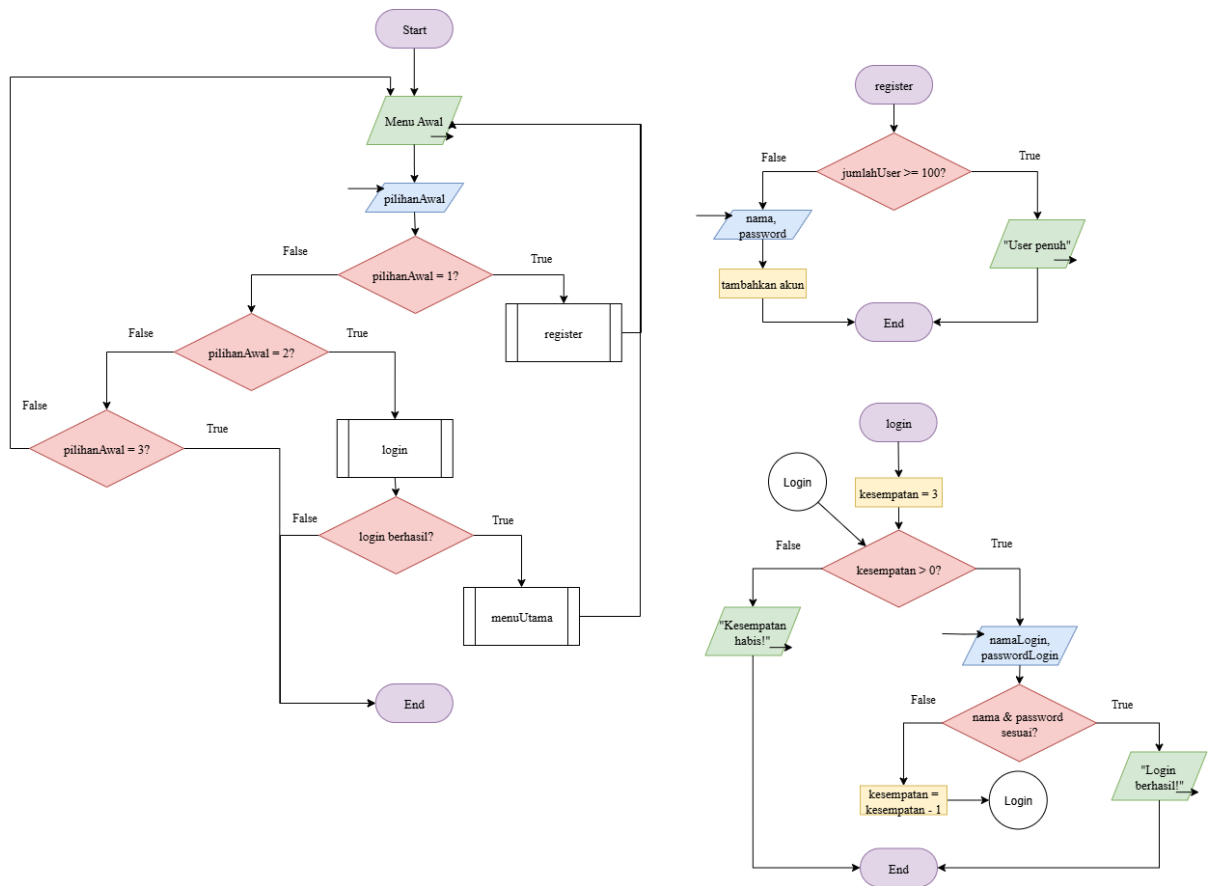
## 7. Implementasi Algoritma Searching

- **Cari Angka Menggunakan Binary Search**

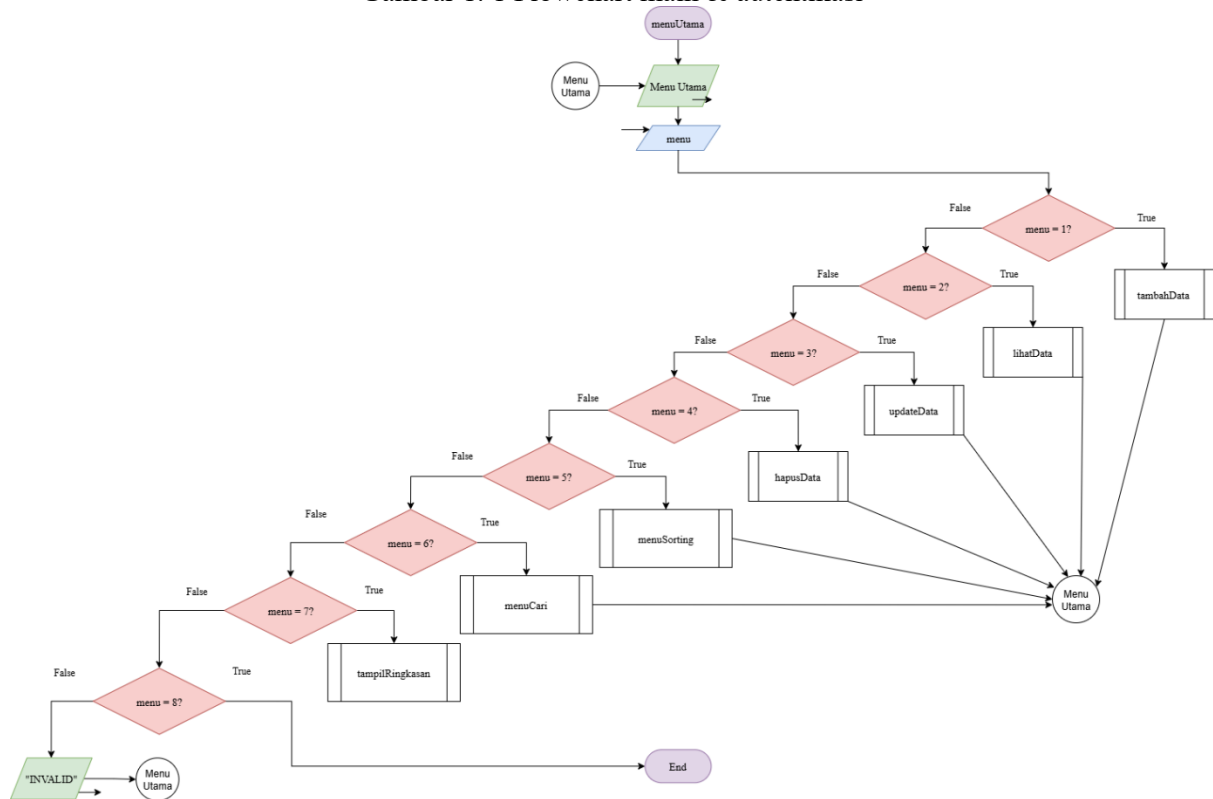
Digunakan untuk mencari ID Kamar. Algoritma ini bekerja pada data yang telah terurut (menggunakan insertionSortIDAsc terlebih dahulu) untuk membagi ruang pencarian menjadi dua secara terus-menerus.

- **Cari Kata Menggunakan Linear Search**

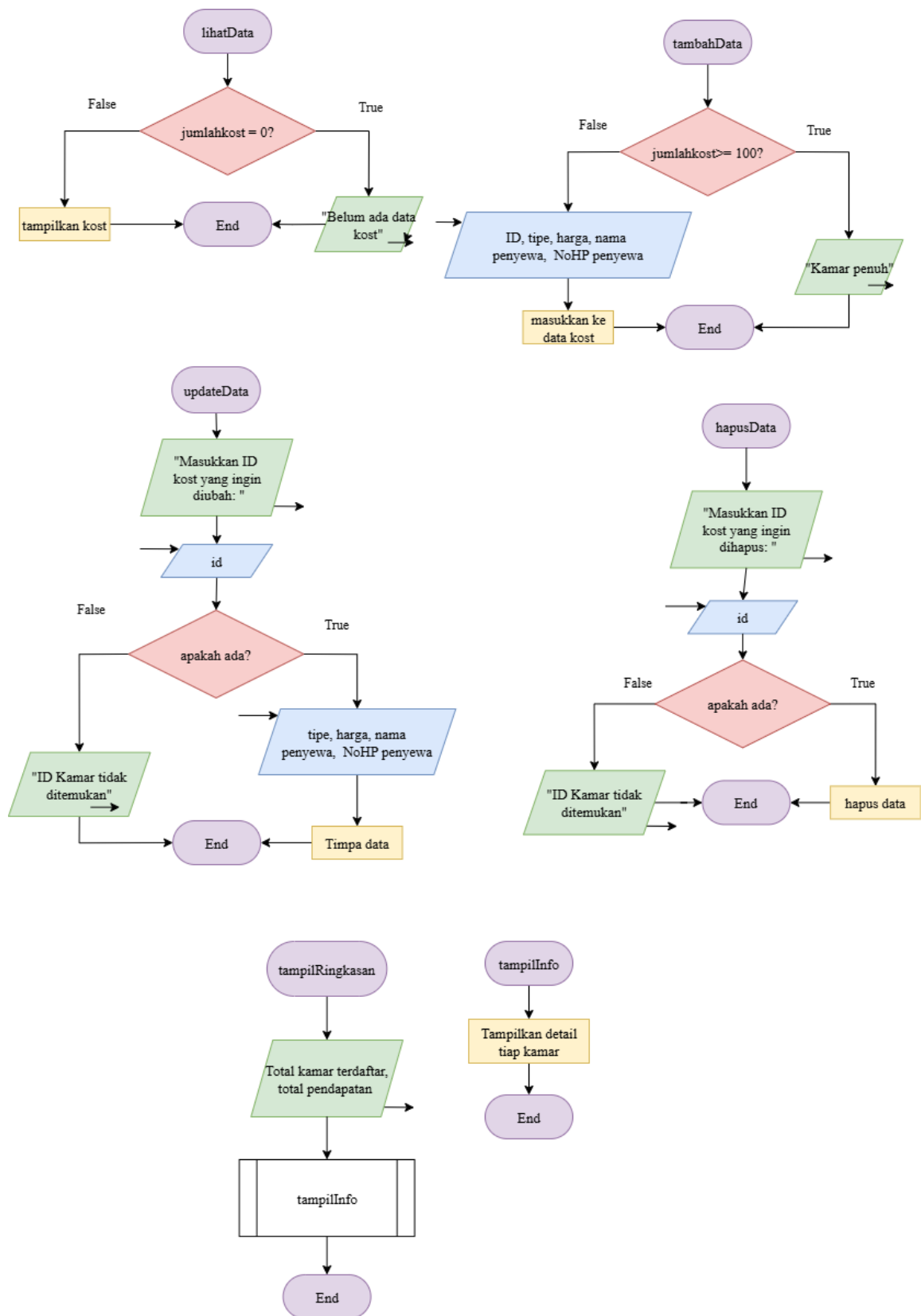
Digunakan untuk mencari Nama Penyewa. Algoritma ini memeriksa setiap elemen satu per satu dari awal hingga akhir, sangat efektif untuk pencarian string pada data yang tidak terurut.



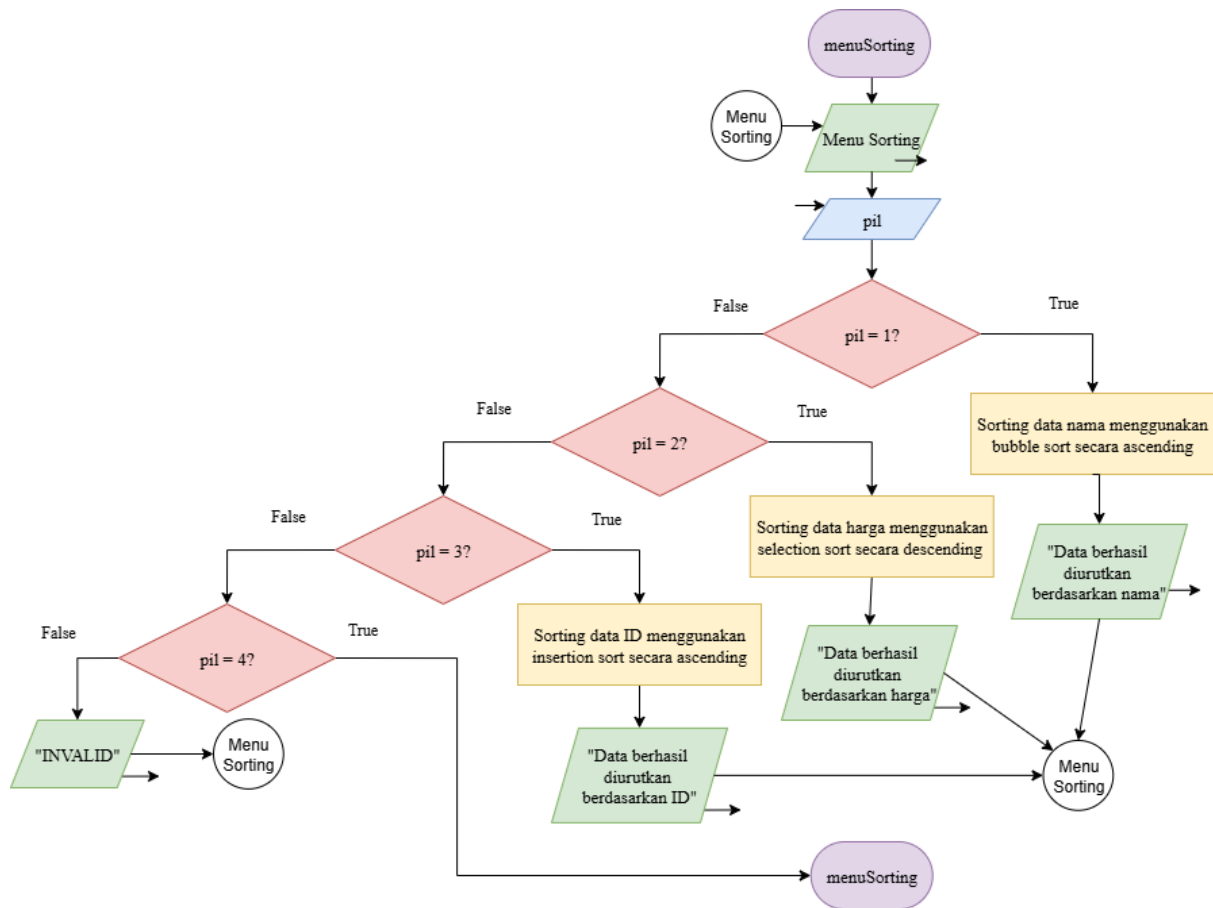
Gambar 1. 1 Flowchart main & autentikasi



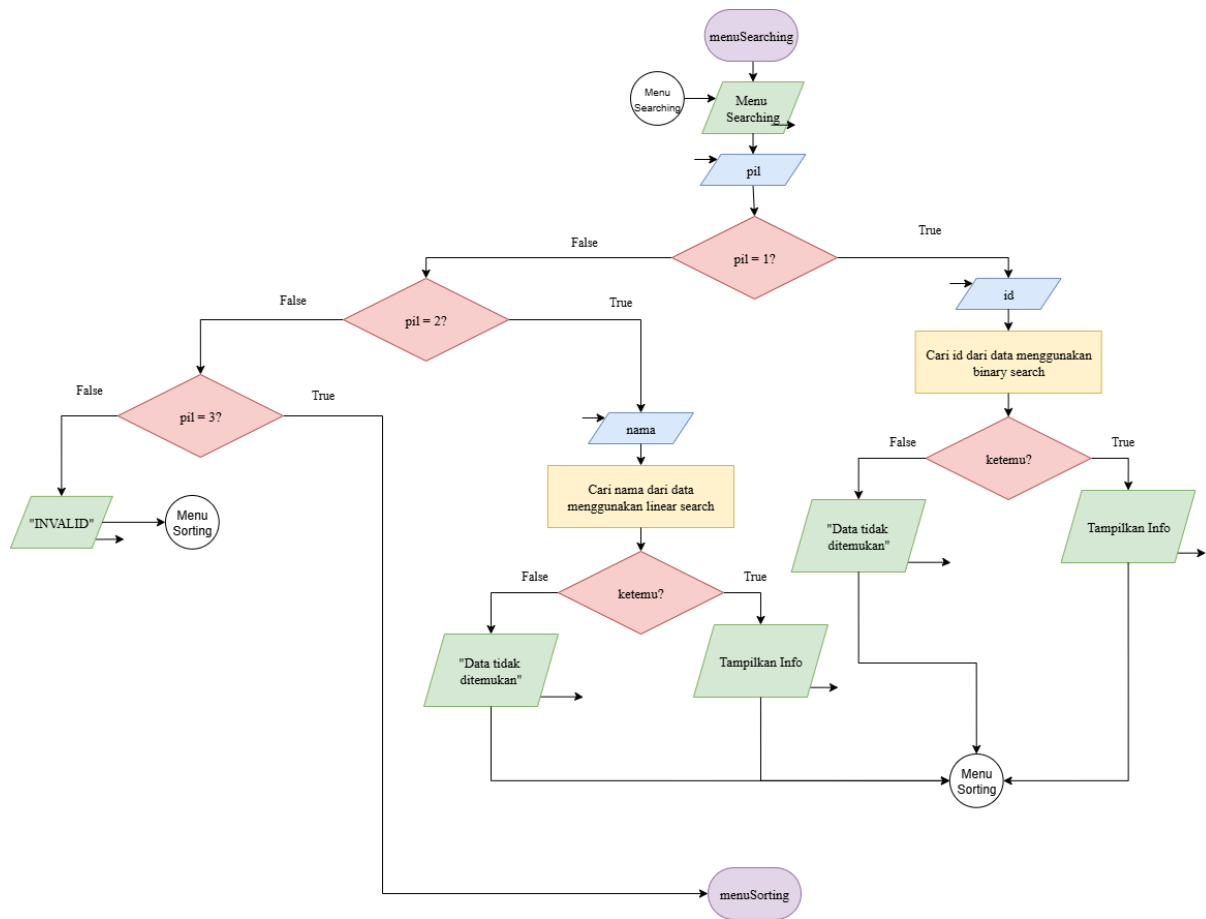
Gambar 1. 2 Flowchart Menu Utama



Gambar 1. 3 Flowchart CRUD



Gambar 1. 4 Flowchart Menu Sorting



Gambar 1. 5 Flowchart Menu Searching

## 2. Deskripsi Singkat Program

Program ini dirancang untuk mendigitalisasi pengelolaan data operasional kost agar lebih terstruktur, aman, dan efisien melalui sistem autentikasi Login dan Register. Dengan implementasi fungsi CRUD (Create, Read, Update, Delete), pengelola dapat mengelola data penyewa secara praktis, yang didukung oleh fungsi rekursif untuk pencarian ID dan penghitungan pendapatan total secara otomatis. Optimalisasi program diperkuat dengan penggunaan Pointer dan Pass by Reference untuk efisiensi manajemen memori, serta penerapan berbagai algoritma Sorting (Bubble, Selection, Insertion) dan Searching (Linear dan Binary Search) untuk mempercepat pengorganisasian serta penemuan informasi data penyewa.

## 3. Source Code

### - PENGATURAN STRUKTUR DATA

Tempat mendefinisikan "wadah" data menggunakan struct. User untuk akun, Penyewa untuk profil tamu, dan Kost sebagai data utama yang menggabungkan informasi kamar dan penyewa secara nested (bersarang) untuk menjaga relasi data.

```
struct Penyewa {
    string nama;
    string noHP;
};
struct Kost {
    int idKamar;
    string tipe;
    int harga;
    Penyewa penyewa;
};
struct User {
    string nama;
    string password;
};
```

### - FITUR ANTARMUKA

Berisi prosedur tampilHeader untuk menjaga konsistensi visual program. Prosedur ini memastikan setiap perpindahan menu memiliki identitas yang jelas dan rapi.

```
void tampilHeader(string judul) {
    cout << "\n==== " << judul << " =====\n";
}
```

### - FITUR AUTENTIKASI

Fitur ini bertindak sebagai gerbang masuk keamanan. Menggunakan parameter pass-by-reference (&) pada registerUser untuk efisiensi. Terdapat sistem pembatasan login maksimal 3 kali untuk mencegah upaya brute-force atau menebak kata sandi secara ilegal.

```

bool registerUser(User users[], int &jumlahUser) {
    tampilHeader("REGISTER");
    if (jumlahUser >= 100) {
        cout << "Kapasitas user penuh!\n";
        return false;
    }
    cin.ignore(1000, '\n');
    cout << "Username: ";
    getline(cin, users[jumlahUser].nama);
    cout << "Password: ";
    getline(cin, users[jumlahUser].password);

    jumlahUser++;
    cout << "Registrasi berhasil!\n";
    return true;
}

bool loginUser(User users[], int jumlahUser) {
    tampilHeader("LOGIN");
    string namaLogin, passwordLogin;
    int kesempatan = 3;
    while (kesempatan > 0) {
        cin.ignore(1000, '\n');
        cout << "Username      : ";
        getline(cin, namaLogin);
        cout << "Password: ";
        getline(cin, passwordLogin);

        for (int i = 0; i < jumlahUser; i++) {
            if (users[i].nama == namaLogin && users[i].password == passwordLogin) {
                cout << "Login berhasil! Selamat datang, " << namaLogin << "!\n";
                return true;
            }
        }
        kesempatan--;
        if (kesempatan > 0) {
            cout << "Login gagal! Sisa percobaan: " << kesempatan << "\n\n";
        }
    }
    cout << "Kesempatan habis! Program berhenti.\n";
    return false;
}

```

## - FITUR PENCARIAN & PERHITUNGAN

Menggunakan logika Rekursif dan Searching tingkat lanjut. cariKamar menelusuri data secara rekursif untuk pencarian ID presisi, sementara totalHarga menjumlahkan seluruh harga kamar secara otomatis. Program juga menerapkan Binary Search (pencarian biner) untuk pencarian angka ID yang lebih cepat dan Linear Search untuk pencarian string nama, di mana keduanya menggunakan Pointer sebagai parameter untuk optimasi memori.

```

int cariKamar(Kost dataKost[], int id, int index, int jumlahKost) {
    if (index >= jumlahKost) return -1;
    if (dataKost[index].idKamar == id) return index;
    return cariKamar(dataKost, id, index + 1, jumlahKost);
}

int totalHarga(Kost dataKost[], int index, int jumlahKost) {
    if (index >= jumlahKost) return 0;
    return dataKost[index].harga + totalHarga(dataKost, index + 1, jumlahKost);
}

```

```

int binarySearchID(Kost dataKost[], int n, int *target) {
    insertionSortIDAsc(dataKost, n);
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (dataKost[mid].idKamar == *target) return mid;
        if (dataKost[mid].idKamar < *target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int linearSearchNama(Kost dataKost[], int n, string *target) {
    for (int i = 0; i < n; i++) {
        if (dataKost[i].penyewa.nama == *target) {
            return i;
        }
    }
    return -1;
}

```

#### - **FITUR UTAMA MANAJEMEN DATA**

Mengelola siklus hidup data dengan pemanfaatan Pointer dan Reference untuk manipulasi data langsung pada memori asli:

- Alokasi data baru dengan validasi kapasitas array.
- Penyajian data terstruktur dalam format tabel menggunakan setw.
- Melakukan modifikasi pada elemen data spesifik berdasarkan hasil pencarian ID, memastikan informasi tetap aktual tanpa mengubah struktur array.
- Menghapus data terpilih berdasarkan hasil pencarian ID, memastikan informasi tetap aktual tanpa mengubah struktur array

```

bool tambahData(Kost dataKost[], int &jumlahKost) {
    tampilHeader("TAMBAH DATA");
    if (jumlahKost >= 100) {
        cout << "Kapasitas kamar penuh!\n";
        return false;
    }
    dataKost[jumlahKost].idKamar = jumlahKost + 1;
    cout << "ID Kamar1: " << dataKost[jumlahKost].idKamar << endl;
    cin.ignore(1000, '\n');
    cout << "Tipe Kamar    : "; getline(cin, dataKost[jumlahKost].tipe);
    cout << "Harga          : "; cin >> dataKost[jumlahKost].harga;
    cin.ignore(1000, '\n');
    cout << "Nama Penyewa   : "; getline(cin, dataKost[jumlahKost].penyewa.nama);
    cout << "No HP Penyewa : "; getline(cin, dataKost[jumlahKost].penyewa.noHP);
    jumlahKost++;
    cout << "Data berhasil ditambahkan!\n";
    return true;
}

void lihatData(Kost dataKost[], int jumlahKost) {
    tampilHeader("DATA KOST");
    if (jumlahKost == 0) {
        cout << "Belum ada data kost.\n";
        return;
    }
    cout << left
         << setw(5)  << "ID"
         << setw(12) << "Tipe"
         << setw(12) << "Harga"
         << setw(20) << "Nama Penyewa"
         << setw(15) << "No HP"
         << endl;
    cout << string(64, '-') << endl;
    for (int i = 0; i < jumlahKost; i++) {
        cout << left
             << setw(5)  << dataKost[i].idKamar
             << setw(12) << dataKost[i].tipe
             << setw(12) << dataKost[i].harga
             << setw(20) << dataKost[i].penyewa.nama
             << setw(15) << dataKost[i].penyewa.noHP
             << endl;
    }
}

```

```

bool updateData(Kost dataKost[], int jumlahKost) {
    tampilHeader("UPDATE DATA");
    int id;
    cout << "Masukkan ID Kamar yang ingin diubah: ";
    cin >> id;

    int index = cariKamar(dataKost, id, 0, jumlahKost);
    if (index == -1) {
        cout << "ID Kamar tidak ditemukan!\n";
        return false;
    }

    cin.ignore(1000, '\n');
    cout << "Tipe Baru: ";
    getline(cin, dataKost[index].tipe);
    cout << "Harga Baru: ";
    cin >> dataKost[index].harga;
    cin.ignore(1000, '\n');
    cout << "Nama Penyewa Baru: ";
    getline(cin, dataKost[index].penyewa.nama);
    cout << "No HP Baru: ";
    getline(cin, dataKost[index].penyewa.noHP);
    cout << "Data berhasil diupdate!\n";
    return true;
}

bool hapusData(Kost dataKost[], int &jumlahKost) {
    tampilHeader("HAPUS DATA");
    int id;
    cout << "Masukkan ID Kamar yang ingin dihapus: ";
    cin >> id;

    int index = cariKamar(dataKost, id, 0, jumlahKost);
    if (index == -1) {
        cout << "ID Kamar tidak ditemukan!\n";
        return false;
    }

    for (int j = index; j < jumlahKost - 1; j++) {
        dataKost[j] = dataKost[j + 1];
    }
    jumlahKost--;
    cout << "Data berhasil dihapus!\n";
    return true;
}

```

#### - FITUR PENGURUTAN

Implementasi tiga metode pengurutan berbeda untuk organisasi data yang lebih dinamis berdasarkan kebutuhan pengguna (Abjad, Harga, atau ID).

```

void bubbleSortNamaAsc(Kost dataKost[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (dataKost[j].penyewa.nama > dataKost[j + 1].penyewa.nama) {
                Kost temp = dataKost[j];
                dataKost[j] = dataKost[j + 1];
                dataKost[j + 1] = temp;
            }
        }
    }
}

void selectionSortHargaDesc(Kost dataKost[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int max_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (dataKost[j].harga > dataKost[max_idx].harga) {
                max_idx = j;
            }
        }
        Kost temp = dataKost[max_idx];
        dataKost[max_idx] = dataKost[i];
        dataKost[i] = temp;
    }
}

void insertionSortIDAsc(Kost dataKost[], int n) {
    for (int i = 1; i < n; i++) {
        Kost key = dataKost[i];
        int j = i - 1;
        while (j >= 0 && dataKost[j].idKamar > key.idKamar) {
            dataKost[j + 1] = dataKost[j];
            j = j - 1;
        }
        dataKost[j + 1] = key;
    }
}

```

## - FITUR INFORMASI & RINGKASAN

Penerapan Function Overloading dan Pointer pada Struct. Menggunakan nama fungsi yang sama (tampilInfo) namun dengan parameter berbeda untuk menampilkan kedalaman informasi yang fleksibel, serta akses data melalui alamat memori (address-of operator &).

```
void tampilInfo(Kost *k) {
    cout << "[Info Kamar] ID: " << k->idKamar << " | Tipe: " << k->tipe << endl;
}
void tampilInfo(Kost *k, bool tampilHarga) {
    cout << "[Info Kamar] ID: " << k->idKamar
        << " | Tipe: " << k->tipe;
    if (tampilHarga)
        cout << " | Harga: Rp" << k->harga;
    cout << " | Penyewa: " << k->penyewa.nama
        << " | No HP: " << k->penyewa.noHP << endl;
}
void tampilInfo(Kost dataKost[], int index) {
    Kost *ptr = &dataKost[index];
    cout << "[Info Kamar ke-" << index + 1 << "]" << "
        << "ID: " << ptr->idKamar
        << " | Tipe: " << ptr->tipe
        << " | Harga: Rp" << ptr->harga
        << " | Penyewa: " << ptr->penyewa.nama << endl;
}
```

## - PROGRAM UTAMA

Pusat kendali alur program yang menggunakan perulangan untuk menjaga stabilitas sesi pengguna. Fitur ini memastikan program tidak berhenti sebelum pengguna memberikan instruksi keluar secara eksplisit.

```

int main() {
    User users[100];
    Kost dataKost[100];
    int jumlahUser = 0;
    int jumlahKost = 0;
    int pilihanAwal;
    do {
        tampilHeader("MENU AWAL");
        cout << "1. Register\n";
        cout << "2. Login\n";
        cout << "3. Keluar\n";
        cout << "Pilihan: ";
        cin >> pilihanAwal;
        if (pilihanAwal == 1) {
            registerUser(users, jumlahUser);
        }
        else if (pilihanAwal == 2) {
            bool berhasil = loginUser(users, jumlahUser);
            if (!berhasil) {
                cout << "Tekan Enter untuk keluar...";
                cin.ignore();
                cin.get();
                return 0;
            }
            menuUtama(dataKost, jumlahKost);
        }
        else if (pilihanAwal != 3) {
            cout << "Pilihan tidak valid!\n";
        }
    } while (pilihanAwal != 3);
    cout << "\nProgram selesai. Terima kasih!\n";
    cout << "Tekan Enter untuk keluar...";
    cin.ignore();
    cin.get();
    return 0;
}

```

## 4. Hasil Output

```
===== MENU AWAL =====
1. Register
2. Login
3. Keluar
Pilihan: 1

===== REGISTER =====
Username: Admin_Kost
Password: 123
Registrasi berhasil!

===== MENU AWAL =====
1. Register
2. Login
3. Keluar
Pilihan: 2

===== LOGIN =====
Username      : Admin_Kost
Password: 123
Login berhasil! Selamat datang, Admin_Kost!
```

Gambar 4. 1 Output register & login

```
===== TAMBAH DATA =====
ID: 1
Tipe Kamar      : Eksklusif
Harga           : 2500000
Nama Penyewa    : Budi
No HP Penyewa   : 0812345
Data berhasil ditambahkan!
```

Gambar 4. 2 Output tambah data

```
===== DATA KOST =====
ID  Tipe      Harga      Nama Penyewa    No HP
-----
1   Eksklusif  2500000  Budi           0812345
2   Reguler    1500000  Caca           1500000
3   VIP        3000000  Andi           08333333
4   Besar      1750000  Alomani        08444444
```

Gambar 4. 3 Output lihat data

```
===== UPDATE DATA =====
Masukkan ID Kamar yang ingin diubah: 2
Tipe Baru: Reguler
Harga Baru: 1500000
Nama Penyewa Baru: Caca
No HP Baru: 08111111
Data berhasil diupdate!
```

Gambar 4. 4 Output update data

```

===== HAPUS DATA =====
Masukkan ID Kamar yang ingin dihapus: 5
Data berhasil dihapus!

```

Gambar 4. 5 Output hapus data

```

===== MENU SORTING =====
1. Urutkan Nama Penyewa (Ascending)
2. Urutkan Harga Kamar (Descending)
3. Urutkan ID Kamar (Ascending)
4. Kembali ke Menu Utama

```

Gambar 4. 6 Tampilan awal menu sorting

```

===== MENU SORTING =====
1. Urutkan Nama Penyewa (Ascending)
2. Urutkan Harga Kamar (Descending)
3. Urutkan ID Kamar (Ascending)
4. Kembali ke Menu Utama
Pilihan: 1
Data berhasil diurutkan berdasarkan nama.
===== DATA KOST =====

```

| ID | Tipe      | Harga   | Nama Penyewa | No HP    |
|----|-----------|---------|--------------|----------|
| 4  | Besar     | 1750000 | Alomani      | 08444444 |
| 3  | VIP       | 3000000 | Andi         | 08333333 |
| 1  | Eksklusif | 2500000 | Budi         | 0812345  |
| 2  | Reguler   | 1500000 | Caca         | 08111111 |

Gambar 4. 7 Output menu sorting 1 & lihat data kembali

```

===== MENU SORTING =====
1. Urutkan Nama Penyewa (Ascending)
2. Urutkan Harga Kamar (Descending)
3. Urutkan ID Kamar (Ascending)
4. Kembali ke Menu Utama
Pilihan: 2
Data berhasil diurutkan berdasarkan harga.
===== DATA KOST =====

```

| ID | Tipe      | Harga   | Nama Penyewa | No HP    |
|----|-----------|---------|--------------|----------|
| 3  | VIP       | 3000000 | Andi         | 08333333 |
| 1  | Eksklusif | 2500000 | Budi         | 0812345  |
| 4  | Besar     | 1750000 | Alomani      | 08444444 |
| 2  | Reguler   | 1500000 | Caca         | 08111111 |

Gambar 4. 8 Output menu sorting 2 & lihat kembali

```

===== MENU SORTING =====
1. Urutkan Nama Penyewa (Ascending)
2. Urutkan Harga Kamar (Descending)
3. Urutkan ID Kamar (Ascending)
4. Kembali ke Menu Utama
Pilihan: 3
Data berhasil diurutkan berdasarkan ID.

===== DATA KOST =====
ID   Tipe      Harga      Nama Penyewa      No HP
-----
1    Eksklusif  2500000    Budi              0812345
2    Reguler    1500000    Caca              08111111
3    VIP        3000000    Andi              08333333
4    Besar     1750000    Alomani           08444444

```

Gambar 4. 9 Output menu sorting 3 & lihat kembali

```

===== MENU SEARCHING =====
1. Cari ID Kamar
2. Cari Nama Penyewa
Pilihan: 1
Masukkan ID yang dicari: 3
[Info Kamar] ID: 3 | Tipe: VIP | Harga: Rp3000000 | Penyewa: Andi | No HP: 08333333

```

Gambar 4. 10 Output menu searching 1

```

===== MENU SEARCHING =====
1. Cari ID Kamar
2. Cari Nama Penyewa
Pilihan: 2
Masukkan Nama yang dicari: Caca
[Info Kamar] ID: 2 | Tipe: Reguler | Harga: Rp1500000 | Penyewa: Caca | No HP: 08111111

```

Gambar 4. 11 Output menu searching 2

```

===== MENU AWAL =====
1. Register
2. Login
3. Keluar
Pilihan: 2

===== LOGIN =====
Username      : salah
Password: ga benar
Login gagal! Sisa percobaan: 2

Username      : salah
Password: ga benar
Login gagal! Sisa percobaan: 1

Username      : salah
Password: ga benar
Kesempatan habis! Program berhenti.
Tekan Enter untuk keluar...

```

Gambar 4. 12 Output login gagal hingga 3 kali

## 5. Langkah-langkah GIT

### 5.1 GIT add

Perintah “git add” digunakan untuk menambahkan file ke staging area sebelum dilakukan commit. Pada gambar yang digunakan “git add .” agar semua perubahan di folder saat ini langsung ditambahkan sekaligus, tanpa harus memilih file satu per satu.

```
● PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl\post-test\post-test-apl-6> git add .
```

Gambar 5. 1 git add

### 5.2 GIT Commit

Perintah “git commit” digunakan untuk menyimpan perubahan dari staging area ke riwayat repository lokal. Pada gambar digunakan opsi -m, dipakai untuk memberi pesan singkat.

```
● PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl\post-test\post-test-apl-6> git commit -m "Finis POSTTEST 6"
[main 7e8a214] Finis POSTTEST 6
3 files changed, 362 insertions(+)
create mode 100644 post-test/post-test-apl-6/2509106073-HAMMAMSYAMIL-PT-6.cpp
create mode 100644 post-test/post-test-apl-6/2509106073-HAMMAMSYAMIL-PT-6.exe
create mode 100644 post-test/post-test-apl-6/2509106073_HAMMAMSYAMIL-PT-6.pdf
```

Gambar 5. 2 git commit

### 5.3 GIT Push

Perintah “git push” digunakan untuk mengirim commit dari repository lokal ke repository remote.

```
● PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl\post-test\post-test-apl-6> git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.75 MiB | 540.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Aeloxyll/praktikum-apl
8786a08..7e8a214 main -> main
```

Gambar 5. 3 git push